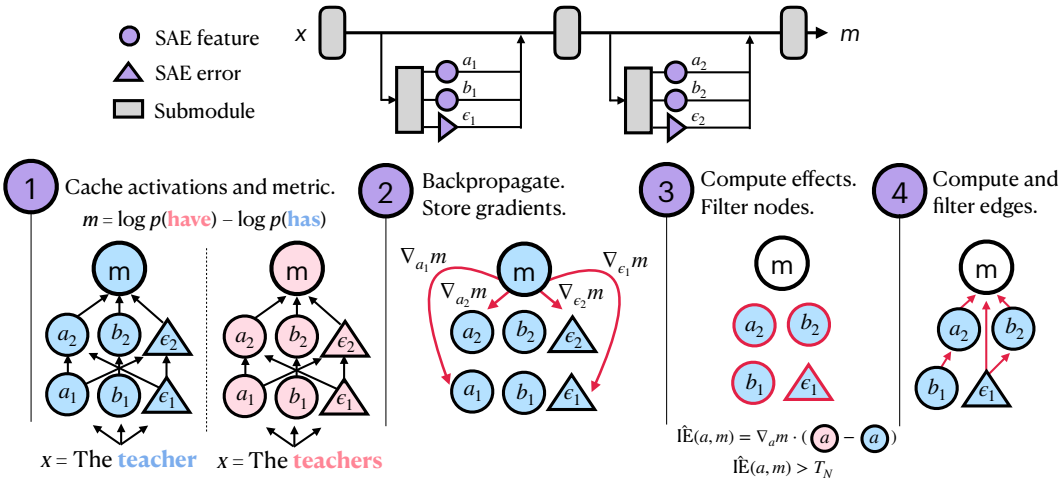

Figure 2: Overview of our method. We view our model as a computation graph that includes SAE features and errors. We cache activations (Step 1) and compute gradients (Step 2) for each node. We then compute approximate indirect effects with Eq. (3; shown) or (4) and filter according to a node threshold T_N (Step 3). We similarly compute and filter edges (Step 4); see App. A.1.

3 Sparse Feature Circuit Discovery

3.1 Method

Suppose we are given an LM M, SAEs for various submodules of M (e.g., attention outputs, MLP outputs, and residual stream vectors, as in §2), a dataset $\mathcal{D}$ consisting either of contrastive pairs $(x_{\text{clean}}, x_{\text{patch}})$ of inputs or of single inputs x, and a metric m that depends on M's output when processing data from $\mathcal{D}$. For example, Figure 2 shows the case where $\mathcal{D}$ consists of pairs of inputs which differ in number, and m is the log probability difference between M outputting the verb form that is correct for the patch vs. clean input.

Viewing SAE features as part of the model. A key idea underpinning our method is that, by applying the decomposition (1) to various hidden states $\mathbf{x}$ in the LM, we can view the feature activations f_i and SAE errors $\boldsymbol{\epsilon}$ as being part of the LM's computation. We can thus represent the model as a computation graph G where nodes correspond to feature activations or SAE errors at particular token positions.

Approximating the IE of each node. Let $\hat{\text{IE}}$ be one of $\hat{\text{IE}}_{\text{atp}}$ or $\hat{\text{IE}}_{\text{ig}}$ (see §2). Then for each node $\mathbf{a}$ in G and input $x \sim \mathcal{D}$, we compute $\hat{\text{IE}}(m; \mathbf{a}; x)$; we then average over $x \sim \mathcal{D}$ to produce a score $\hat{\text{IE}}(m; \mathbf{a})$ and filter for nodes with $|\hat{\text{IE}}(m; \mathbf{a})| > T_N$ for some choice T_N of node threshold.

Consistent with prior work (Nanda, 2022; Kramár et al., 2024), we find that $\hat{\text{IE}}_{\text{atp}}$ accurately estimates IEs for SAE features and SAE errors, with the exception of nodes in the layer 0 MLP and early residual stream layers, where $\hat{\text{IE}}_{\text{atp}}$ underestimates the true IE. We find that $\hat{\text{IE}}_{\text{ig}}$ significantly improves accuracy over $\hat{\text{IE}}_{\text{atp}}$ for these components, so we use it in our experiments below. See Appendix H for more information about linear approximation quality.

Approximating the IE of edges. Using an analogous linear approximation, we also compute the average IE of edges in the computation graph. Although the idea is simple, the mathematics are somewhat involved, so we relegate the details to App. A.1. After computing these IEs, we filter for edges with absolute IE exceeding some edge threshold T_E.

Aggregation across token positions and examples. For templatic data where tokens in matching positions play consistent roles (see §3.2, 3.3), we take the mean effect of nodes/edges across examples. For non-templatic data (§4, 5) we first sum the effects of corresponding nodes/edges across token position before taking the example-wise mean. See App. A.2.

Structure	Example *clean* input	Example output
Simple	The **parents**	p(**is**) $-$ p(**are**)
Within RC	The athlete that the **managers**	p(**likes**) $-$ p(**like**)
Across RC	The **athlete** that the managers like	p(**do**) $-$ p(**does**)
Across PP	The **secretaries** near the cars	p(**has**) $-$ p(**have**)

Table 1: Example clean inputs x and outputs m for subject-verb agreement tasks.

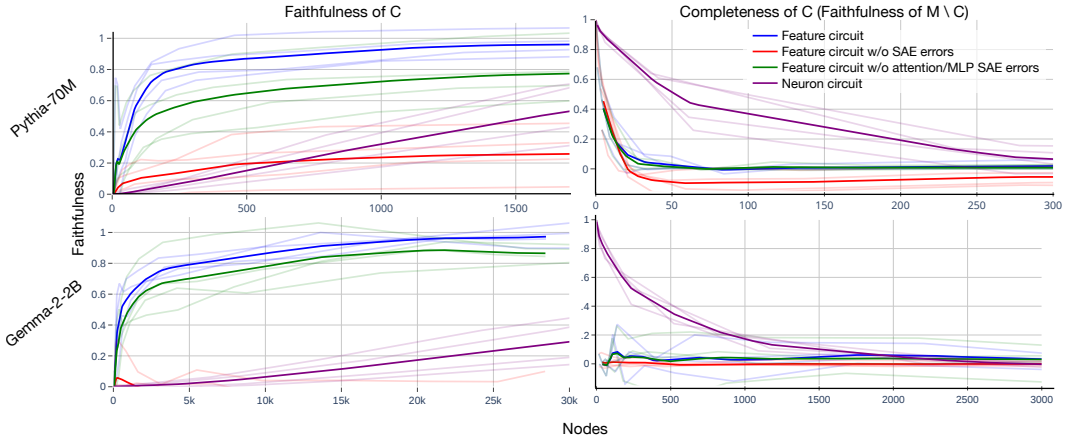

Figure 3: Faithfulness and completeness scores for circuits, measured on held-out data. Faint lines correspond to the structures from Table 1, with the average across structures in bold. The ideal faithfulness for circuits is 1, while the ideal completeness is 0.

Practical considerations. Various practical difficulties arise for efficiently computing the gradients needed by our method. We solve these using a combination of stop gradients, pass-through gradients, and tricks for efficient Jacobian-vector product computation; see App. A.3.

3.2 Discovering and Evaluating Sparse Feature Circuits for Subject-Verb Agreement

To evaluate our method, we discover sparse feature circuits (henceforth, feature circuits) on Pythia-70M and Gemma-2-2B for four variants of the subject-verb agreement task (Table 1). Specifically, we adapt data from Finlayson et al. (2021) to produce datasets consisting of contrastive pairs of inputs that differ only in the grammatical number of the subject; the model's task is to choose the appropriate verb inflection.

We evaluate circuits for **interpretability**, **faithfulness**, and **completeness**. For each criterion, we compare to *neuron* circuits discovered by applying our methods with neurons in place of sparse features; in this setting, there are no error terms $\boldsymbol{\epsilon}$. When evaluating feature circuits for faithfuless and completeness, we use a test split of our dataset, consisting of contrastive pairs not used to discover the circuit.

Interpretability. For Pythia SAEs, we asked human crowdworkers to rate the interpretability of random features, random neurons, features from our feature circuits, and neurons from our neuron circuits. Crowdworkers rated sparse features as significantly more interpretable than neurons, with features that participate in our circuits also being more interpretable than randomly sampled ones (App. F). This replicates prior findings that SAE features are substantially more interpretable than neurons (Bricken et al., 2023). For Gemma-2 SAEs, we refer the reader to Lieberum et al. (2024), which finds the interpretability of these SAEs' features to be on par with those trained via other state-of-the-art techniques.

Faithfulness. Given a circuit C and metric m, let $m(C)$ denote the average value of m over inputs from $\mathcal{D}$ when running our model with all nodes outside of C mean-ablated, i.e., set to their average

value over data from $\mathcal{D}$.[2] We then measure the faithfulness of a circuit as $\frac{m(C)-m(\varnothing)}{m(M)-m(\varnothing)}$, where $\varnothing$ denotes the empty circuit and M denotes the full model. Intuitively, this metric captures the proportion of the model's performance our circuit explains, relative to mean ablating the full model (which represents the "prior" performance of the model when it is given information about the task, but not about specific inputs).

We find that components in early model layers are typically involved in processing specific tokens. In practice, the inputs in the train split of our dataset (used to discover the circuit) and the test split (for evaluation) do not contain identical tokens, making it difficult to evaluate the quality of early segments of our circuit. Thus, we ignore the first $1/3$ of our circuit, and only evaluate the latter $2/3$.

We plot faithfulness for feature circuits and neuron circuits after sweeping over node thresholds T_N (Fig. 3). We find that small feature circuits explain a large proportion of model behavior: the majority of performance in Pythia-70M, resp. Gemma-2-2B is explained by only 100, resp. 500 nodes. In contrast, around 1500, resp. 50000 neurons are required to explain half the performance. However, as SAE error nodes are high-dimensional and coarse-grained, they cannot be fairly compared to neurons; we thus also plot the faithfulness of feature circuits with all SAE error nodes removed, or with all attention and MLP error nodes removed. Unsurprisingly, we find that removing residual stream SAE error nodes severely disrupts the model and curtails its maximum performance; removing MLP and attention error nodes is less disruptive.

Completeness. Are there parts of the model behavior that our circuit fails to capture? We measure this as the faithfulness of the circuit's complement $M \setminus C$ (Fig. 3). We observe that we can eliminate the model's task performance by ablating only a few nodes from our feature circuits, and that this is true even when we leave all SAE errors in place. In contrast, it takes hundreds (for Pythia) or thousands (for Gemma) of neurons to achieve the same effect.

3.3 Case study: Subject-verb agreement across a relative clause

We find that inspecting small feature circuits produced by our technique can provide insights into how Pythia-70M and Gemma-2-2B arrive at observed behaviors. To illustrate this, we present a case study of relatively small feature circuits for subject-verb agreement across a relative clause (RC).

To keep the number of nodes we need to annotate manageable, we tune our node threshold to produce a small circuit with faithfulness > 0.2. For Pythia, this results in a circuit with 86 nodes and faithfulness 0.21; for Gemma we study a circuit with 223 nodes and faithfulness 0.21. We summarize these circuits in Figure 4; the full circuits (as well as small circuits for other subject-verb agreement tasks) can be found in App. C.1. We depict SAE features with rectangles and SAE errors with triangles.

Our circuits depict interpretable algorithms wherein both models of study select appropriate verb forms via two pathways. The first pathway consists of features which detect the number of the main subject and then generically promote matching verb forms. The second pathway begins the same, but moves the relevant number information to the end of the relative clause by using PP/RC boundary detectors. Gemma 2 also uses noun phrase (NP) number trackers, which detect the number of the noun that heads an NP and remain active on all tokens until the end of the NP; these promote matching verb forms at each position, but especially at the last token of an NP.

We find significant overlap between this circuit and the circuit we discovered for agreement across a prepositional phrase, with Pythia-70M and Gemma-2-2B handling these syntactically distinct structures in a mostly uniform way. In accordance with Finlayson et al. (2021), we find less overlap with our circuits for simple agreement and within RC agreement (Appendix C.1).

4 Application: Removing unintended signals from a classifier without disambiguating labels

NN classifiers often rely on unintended signals—e.g., spurious features. Nearly all prior work on mitigating this problem relies on access to *disambiguating labeled data* in which unintended signals are less predictive of labels than intended ones. However, some tasks have structural properties

[2]Following Wang et al. (2023), we ablate features by setting them to their mean *position-specific* values.